Aula técnica: entendendo o script Player

27/10/2026 · Instrutor Pedro Leonardo

O que vamos ver hoje

O script Player já funciona: o personagem anda, corre, freia, pula e gira a câmera. Hoje a gente não vai criar nada novo. Vamos abrir o script e entender o que cada parte faz, para conseguir mudar e criar coisas parecidas sozinho.

- **Conceitos de C#**: as palavras do código que aparecem o tempo todo.
- **Enum**: a lista de estados do movimento.
- **Variáveis**: as configurações do Inspector e as anotações do script.
- **Funções**: o que acontece quando o jogo começa, a cada frame e a cada passo da física.

A ordem em que a Unity roda o script:

Start (uma vez) → Update (a cada frame) → FixedUpdate (50 vezes por segundo)

Dica: deixe o script aberto ao lado deste documento e procure cada trecho no código enquanto lê.

Etapa 1: conceitos de C#

Por quê: algumas palavras aparecem no script inteiro. Entendendo elas primeiro, o resto fica bem mais fácil de ler.

Palavra	O que quer dizer	Exemplo no script
`using`	quais ferramentas o script usa	`using UnityEngine;`
`class`	o molde do script, com variáveis e funções	`public class Player`
`MonoBehaviour`	permite colocar o script num objeto da cena	`Player : MonoBehaviour`
`public`	aparece no Inspector e outros scripts enxergam	`public float forcaPulo`
`private`	só este script enxerga	`private Rigidbody rb`
`void`	função que não devolve nada	`void Pular()`
`return`	devolve um valor e termina a função	`return 1000f;`

Tipos de variável:

- `int`: número inteiro, como `pulosMaximos = 1`.
- `float`: número com vírgula. O `f` no fim marca o float: `velocidadeAndar = 5f`.

- `bool`: verdadeiro (`true`) ou falso (`false`), como `aumentoGradual = true`.
- `Vector3`: três números (X, Y, Z). Pode ser uma posição, uma direção ou uma velocidade. Na Unity, **Y é para cima**.

Para medir tempo, o script usa `Time.time`, o relógio do jogo em segundos. Ele guarda o momento em que algo aconteceu e depois faz `Time.time - momento` para saber quanto tempo passou.

Etapa 2: o enum EstadoMovimento

Por quê: o player precisa saber em que situação está. Em vez de usar números (0, 1, 2, 3), que ninguém lembra o que significam, usamos uma lista de nomes.

```
// Estados do movimento do player
public enum EstadoMovimento
{
    Parado,
    Andando,
    Correndo,
    Deslizando   // soltou as teclas, mas ainda está freando
}
```

Entendendo o código:

- Um **enum** é uma lista fechada de nomes. Uma variável desse tipo só aceita um valor da lista.
- Para usar um valor, escreva o nome do enum, um ponto e o valor: `EstadoMovimento.Correndo`.
- Escrever um valor errado, como `EstadoMovimento.Corendo`, dá erro na hora. Com números, o erro passaria sem ninguém perceber.
- O enum fica fora da classe Player para que outros scripts, como o PlayerUI, também possam usar.

Estado	Quando acontece
Parado	nenhuma tecla apertada e o player quase sem velocidade
Andando	tecla apertada, mas ainda não deu o tempo de correr
Correndo	andou sem parar até completar o tempo para correr
Deslizando	soltou as teclas, mas ainda está freando

Etapa 3: as configurações do Inspector

Por quê: as variáveis `public` são os "botões de ajuste" do player. Mudando elas no Inspector, o movimento muda sem mexer no código.

As palavras entre colchetes em cima das variáveis só mudam como elas aparecem no Inspector:

- `[Header("Pulo")]`: coloca um título separando os grupos.
- `[Tooltip("...")]`: mostra um texto de ajuda quando o mouse para em cima do nome.
- `[Min(0f)]`: não deixa colocar um número menor que 0.

- `[Range(0f, 1f)]`: troca a caixa de número por uma barrinha de 0 a 1.

```
[Header("Pulo")]
public float forcaPulo = 6f; // força do pulo

[Min(0)]
[Tooltip("Quantidade de pulos. 0 = não pula, 1 = normal, 2 = duplo...")]
public int pulosMaximos = 1;
```

As configurações do script:

Variável	Padrão	Para que serve
`velocidadeAndar`	5	velocidade ao andar
`velocidadeCorrer`	9	velocidade ao correr
`tempoParaCorrer`	1.5	segundos andando até começar a correr
`aumentoGradual`	ligado	a velocidade sobe aos poucos até a de correr
`curvaAceleracao`	1	formato dessa subida (maior que 1 começa devagar)
`exigirTeclaCorrer`	desligado	só corre segurando Shift
`tempoAcelerar`	0.1	segundos para sair do parado e chegar na velocidade de andar
`tempoFrear`	0.1	segundos para frear até parar
`controleNoAr`	0.5	quanto dá para mudar a direção no ar (0 a 1)
`forcaPulo`	6	altura do pulo
`pulosMaximos`	1	quantidade de pulos (2 = pulo duplo)
`tempoCoyote`	0.15	segundos em que ainda dá para pular depois de sair da borda
`coyotePermitePulosExtras`	ligado	depois do pulo de coyote, ainda pode usar os pulos no ar
`cooldownEntrePulos`	0.2	tempo mínimo entre um pulo e outro
`sensibilidadeMouse`	200	velocidade do giro com o mouse
`cameraPersonagem`	vazio	a câmera do player (arraste no Inspector)
`limiteAnguloVertical`	80	quanto dá para olhar para cima e para baixo

Coyote time é a pequena janela em que o player ainda pode pular logo depois de sair de uma borda. O nome vem do Coiote dos desenhos, que só cai alguns instantes depois de sair do penhasco.

Etapa 4: as anotações do script

Por quê: enquanto o jogo roda, o script precisa lembrar algumas coisas: se está no chão, quantos pulos sobraram, quando foi o último pulo. Essas anotações são variáveis `private`.

```
private Rigidbody rb;               // física do player
private int contatosNoChao = 0;     // quantas plataformas o player está tocando
private int pulosNoAr = 0;          // pulos que ainda sobram no ar
private float ultimoTempoNoChao = -999f; // último momento em que tocou o chão
private bool pulouDoChao = false;        // true depois do pulo do chão
private float tempoUltimoPulo = -999f;   // momento do último pulo
private float tempoMovendo = 0f;         // há quanto tempo está andando sem parar
```

Entendendo o código:

- `rb` guarda o Rigidbody do player. É por ele que o script muda a velocidade.
- `contatosNoChao` **conta** as plataformas em vez de usar true ou false. Assim, se o player estiver em cima de duas plataformas e sair de uma, ele continua no chão.
- `ultimoTempoNoChao` e `tempoUltimoPulo` começam em -999 para o script achar que isso aconteceu "há muito tempo". Se começassem em 0, o primeiro pulo do jogo ficaria bloqueado pelo cooldown.
- `tempoMovendo` é um cronômetro: soma o tempo enquanto o player anda e zera quando ele para.

Etapa 5: informações para outros scripts

Por quê: o painel na tela (PlayerUI) precisa saber se o player está no chão, qual a velocidade e qual o estado. Mas as variáveis que guardam isso são `private`, e o painel não consegue enxergar. A solução é criar uma "janela" pública que só **mostra** o valor, sem deixar ninguém mudar.

Vamos começar pela mais simples:

```
public bool EstaNoChao => contatosNoChao > 0;
```

Esta linha é um jeito curto de escrever isto:

```
public bool EstaNoChao
{
    get
    {
        return contatosNoChao > 0;
    }
}
```

Entendendo o código:

- `EstaNoChao` é uma **propriedade**. Outro script lê ela como se fosse uma variável: `player.EstaNoChao`.
- `get` é o que acontece quando alguém lê. Aqui, ele faz a conta `contatosNoChao > 0` e devolve o resultado.
- Se o player está tocando 1 ou mais plataformas, a conta dá `true`. Se está tocando 0, dá `false`.
- A seta `=>` substitui o `get` e o `return` quando a conta cabe numa linha só. As duas formas fazem a mesma coisa.
- Como só existe o `get`, outro script consegue **ler**, mas não consegue **mudar**. Escrever `player.EstaNoChao = true` dá erro.

A próxima é igual, só que devolve uma variável:

```
public EstadoMovimento EstadoAtual => estadoAtual;
```

- `estadoAtual` (minúscula) é a variável `private` que guarda o estado.
- `EstadoAtual` (maiúscula) é a janela pública que mostra esse valor para outros scripts.

A última faz uma conta um pouco maior. No script ela está numa linha só:

```
public float VelocidadeAtual => rb != null ? new Vector3(rb.velocity.x, 0f, rb.velocity.z).magnitude : 0f;
```

Escrita com `if`, ela fica assim:

```
public float VelocidadeAtual
{
    get
    {
        // Se o Rigidbody ainda não existe, a velocidade é 0
        if (rb == null)
            return 0f;

        // Pega a velocidade sem o Y (sem a queda e o pulo)
        Vector3 velocidadeNoChao = new Vector3(rb.velocity.x, 0f, rb.velocity.z);

        // Devolve o tamanho dessa velocidade
        return velocidadeNoChao.magnitude;
    }
}
```

Entendendo o código:

- O `?` e o `:` da versão curta são um `if` escrito numa linha. Lê-se assim: **se** `rb != null`, **então** calcula a velocidade, **senão** devolve `0f`.
- `rb` só existe depois que o `Start` roda. Antes disso, ele é `null` (vazio), e usar ele daria erro. Por isso o teste.
- `new Vector3(rb.velocity.x, 0f, rb.velocity.z)` copia a velocidade do player, trocando o Y por 0. Assim, cair ou pular não conta como andar.
- `.magnitude` é o tamanho do vetor. Numa velocidade, é o quão rápido o player está indo.

`PulosDisponiveis`, `CoyoteRestante` e `CooldownPuloRestante` são propriedades do mesmo tipo: fazem uma conta na hora e devolvem o resultado para o painel mostrar.

Etapa 6: os valores do checkpoint

Por quê: as zonas mudam o player durante a fase (por exemplo, dão pulo duplo). Quando ele morre, a cena recarrega e ele perderia isso. O checkpoint guarda os valores para devolver depois.

```
private class ValoresSalvos
{
    public int pulosMaximos;
    public float velocidadeAndar, velocidadeCorrer, tempoParaCorrer;
    ...
}
```

```
// "static" = não se perde quando a cena recarrega
private static ValoresSalvos valoresDoCheckpoint;   // valores guardados
private static Vector3? checkpointDosValores;       // checkpoint onde os valores
foram guardados
```

Entendendo o código:

- `ValoresSalvos` é uma classe dentro da classe Player. Funciona como uma ficha com um espaço para cada configuração.
- As duas variáveis são `static`: elas continuam existindo mesmo quando a cena recarrega e o player é criado de novo.
- O `?` em `Vector3?` quer dizer "pode ficar vazio". Um Vector3 normal sempre vale pelo menos (0, 0, 0), e isso poderia ser uma posição de verdade. Com o `?`, vazio quer dizer "ainda não tem checkpoint".
- `SalvarValoresNoCheckpoint()` copia os valores do player para a ficha. `RestaurarValoresDoCheckpoint()` faz o caminho contrário.

Etapa 7: Start

Por quê: antes de tudo começar, o script precisa se preparar. A Unity chama o `Start` **uma vez**, quando o player aparece na cena.

```
void Start()
{
    rb = GetComponent<Rigidbody>();

    // Deixa o movimento mais suave na tela
    rb.interpolation = RigidbodyInterpolation.Interpolate;

    // Prende e esconde o cursor do mouse
    Cursor.lockState = CursorLockMode.Locked;
    Cursor.visible = false;

    // Se tem checkpoint salvo, o player nasce nele
    if (CheckpointManager.posicaoCheckpoint.HasValue)
    {
        transform.position = CheckpointManager.posicaoCheckpoint.Value;
        ...
    }
    ...
}
```

Entendendo o código:

- `GetComponent<Rigidbody>()` procura o Rigidbody no mesmo objeto e guarda em `rb`.
- A interpolação deixa o movimento suave: a física roda 50 vezes por segundo, mas a tela desenha mais vezes. A Unity calcula as posições do meio.
- O cursor fica preso no centro e invisível, para o mouse não sair da janela enquanto gira a câmera.
- `HasValue` pergunta se existe um checkpoint salvo. Se existe, o player nasce nele e recebe os valores guardados.

Etapa 8: Update e o pulo

Por quê: o `Update` roda **a cada frame**. É o lugar certo para perceber o espaço apertado, porque esse aviso dura um frame só.

Primeiro, ele recarrega os pulos quando o player está no chão:

```
bool acabouDePular = Time.time - tempoUltimoPulo < tempoIgnorarChaoAposPulo;

if (contatosNoChao > 0 && !acabouDePular)
{
    ultimoTempoNoChao = Time.time;
    pulouDoChao = false;
    pulosNoAr = Mathf.Max(0, pulosMaximos - 1);
}
```

- Logo depois de pular, o pé ainda encosta no chão por um instante. `acabouDePular` evita que o pulo volte na hora.
- No chão, anota o momento (`ultimoTempoNoChao`), libera o pulo do chão e recarrega os pulos de ar.
- Com pulo duplo (2), sobra 1 pulo para o ar. `Mathf.Max(0, ...)` impede número negativo quando os pulos são 0.

Depois, decide se pode pular:

```
bool podePularDoChao = (Time.time - ultimoTempoNoChao <= tempoCoyote) &&
!pulouDoChao;
bool cooldownLiberado = Time.time - tempoUltimoPulo >= cooldownEntrePulos;
```

- `podePularDoChao` não pergunta "está no chão?", e sim "tocou o chão há pouco tempo?". É assim que o coyote time funciona.
- `cooldownLiberado` confere se já passou o tempo mínimo desde o último pulo.

Por último, o pulo:

```
if (Input.GetKeyDown(KeyCode.Space) && cooldownLiberado)
{
    if (podePularDoChao && pulosMaximos > 0)
    {
        // Pulo do chão (ou de coyote)
        Pular();
        pulosNoAr = pulosMaximos - 1;
        pulouDoChao = true;
        ultimoTempoNoChao = -999f; // fecha o coyote time
    }
    else if (pulosNoAr > 0)
    {
        // Pulo no ar
        Pular();
        pulosNoAr--;
    }
}
```

- `GetKeyDown` é true só no frame em que a tecla foi **apertada**. Segurar o espaço não faz pular sem parar.

- Se pode pular do chão, usa o pulo do chão e guarda o que sobra para o ar.
- Se não pode, mas ainda sobram pulos de ar, gasta um (`pulosNoAr--` tira 1).
- Se nenhum dos dois vale, apertar espaço não faz nada.

O `Update` também gira a câmera com o mouse, confere se o player passou por um checkpoint novo e solta o cursor quando aperta ESC.

Etapa 9: FixedUpdate e Pular

Por quê: tudo que mexe na física do Rigidbody deve rodar no ritmo da física. O `FixedUpdate` roda sempre 50 vezes por segundo, então o player anda igual em qualquer computador.

```
void FixedUpdate()
{
    MovimentarTeclas();
}

void Pular()
{
    tempoUltimoPulo = Time.time;

    // Zera a velocidade de queda para todo pulo ter a mesma altura
    Vector3 v = rb.velocity;
    v.y = 0f;
    rb.velocity = v;

    // Empurra o player para cima
    rb.AddForce(Vector3.up * forcaPulo, ForceMode.VelocityChange);
}
```

Entendendo o código:

- `Pular()` anota o momento do pulo, para o cooldown funcionar.
- Zerar a queda é importante no pulo duplo: o segundo pulo acontece enquanto o player cai. Sem zerar, ele sairia bem mais baixo.
- O C# não deixa mudar só o Y da velocidade do Rigidbody direto. Por isso o script copia, muda e devolve.
- `ForceMode.VelocityChange` soma a força direto na velocidade, sem levar em conta a massa do player.

Etapa 10: MovimentarTeclas

Por quê: é aqui que o player anda. A função faz sempre o mesmo caminho: lê as teclas, descobre a velocidade que o player quer ter e chega um pouco mais perto dela.

```
Vector3 direcao = Vector3.zero;

if (Input.GetKey(KeyCode.W))
    direcao += transform.forward;  // frente
if (Input.GetKey(KeyCode.S))
    direcao -= transform.forward;  // trás
...
```

```
Vector3 alvo = direcao.normalized * velocidadeAtual;
Vector3 atual = new Vector3(rb.velocity.x, 0f, rb.velocity.z);
```

- `transform.forward` é a frente **do player**. Como o player gira com o mouse, a frente é sempre para onde a câmera olha.
- Trás é "menos frente" e esquerda é "menos direita". W e S juntos se cancelam.
- `.normalized` deixa a direção com tamanho 1. Sem ele, andar na diagonal (W + D) seria 41% mais rápido.
- `alvo` é a velocidade que o player **quer** ter. `atual` é a que ele **tem**, sem o Y.

Depois, escolhe quão rápido a velocidade pode mudar:

Situação	Usa
nenhuma tecla apertada	a aceleração de frear
apertou para o lado contrário	a mais forte entre acelerar e frear (vira rápido)
andando normalmente	a aceleração de acelerar
no ar	a mesma, vezes `controleNoAr` (menos controle)

A função `TempoParaAceleracao` transforma os segundos do Inspector em aceleração: `velocidadeAndar / tempo`. Com 5 e 0.1, a velocidade muda 50 por segundo.

```
Vector3 novaVelocidade = Vector3.MoveTowards(atual, alvo, aceleracao *
Time.fixedDeltaTime);
novaVelocidade.y = rb.velocity.y;
rb.velocity = novaVelocidade;
```

- `MoveTowards` leva a velocidade atual na direção do alvo, andando no máximo um passo. Se o alvo está perto, chega exatamente nele.
- O passo é a aceleração vezes a duração de um passo de física (0.02 s). Com aceleração 50, a velocidade sobe 1 por passo: em 5 passos (0.1 s) chega em 5.
- O Y original volta antes de aplicar, para o movimento não atrapalhar a gravidade nem o pulo.

Etapa 11: AtualizarEstado

Por quê: esta função decide se o player está parado, andando, correndo ou deslizando, e qual velocidade ele deve buscar.

```
if (!estaMovendo)
{
    tempoMovendo = 0f;
    velocidadeAtual = 0f;

    if (VelocidadeAtual > 0.1f)
        estadoAtual = EstadoMovimento.Deslizando;
    else
        estadoAtual = EstadoMovimento.Parado;
    return;
}
```

```
tempoMovendo += Time.fixedDeltaTime;
float progresso = Mathf.Clamp01(tempoMovendo / tempoParaCorrer);

float t = Mathf.Pow(progresso, curvaAceleracao);
velocidadeAtual = Mathf.Lerp(velocidadeAndar, velocidadeCorrer, t);

estadoAtual = progresso >= 1f ? EstadoMovimento.Correndo :
EstadoMovimento.Andando;
```

Entendendo o código:

- Sem teclas: zera o cronômetro e o alvo vira 0. Se ainda está se mexendo, está **Deslizando**; senão, **Parado**. O `return` termina a função aqui.
- Andando: o cronômetro soma 0.02 s a cada passo.
- `progresso` vai de 0 (começou a andar) até 1 (correndo). `Mathf.Clamp01` não deixa passar de 1.
- `Mathf.Lerp` acha a velocidade entre andar e correr: com 0 é 5, com 1 é 9, com 0.5 é 7.
- Quando o progresso chega em 1, o estado vira **Correndo**.

Tempo andando	Velocidade alvo	Estado
0 s	5	Andando
0.75 s	7	Andando
1.5 s ou mais	9	Correndo

O código real também trata `aumentoGradual` desligado (troca de uma vez) e a tecla de correr. A ideia é a mesma.

Etapa 12: mouse e colisões

Por quê: falta girar a câmera e saber quando o player está tocando o chão.

```
float mouseX = Input.GetAxis("Mouse X") * sensibilidadeMouse * Time.deltaTime;
float mouseY = Input.GetAxis("Mouse Y") * sensibilidadeMouse * Time.deltaTime;

transform.Rotate(Vector3.up * mouseX);

rotacaoVertical -= mouseY;
rotacaoVertical = Mathf.Clamp(rotacaoVertical, -limiteAnguloVertical,
limiteAnguloVertical);
cameraPersonagem.localRotation = Quaternion.Euler(rotacaoVertical, 0f, 0f);
```

- Para os lados, gira o **player inteiro**. A câmera é filha dele e gira junto.
- Para cima e para baixo, gira **só a câmera**. Se o player inclinasse, o W faria ele andar para dentro do chão.
- `Mathf.Clamp` prende o ângulo entre -80 e 80 graus, para não ficar de cabeça para baixo.

```
void OnCollisionEnter(Collision colisao)
{
    if (colisao.gameObject.CompareTag("Plataforma"))
```

```
        contatosNoChao++;
}

void OnCollisionExit(Collision colisao)
{
    if (colisao.gameObject.CompareTag("Plataforma"))
        contatosNoChao = Mathf.Max(0, contatosNoChao - 1);
}
```

- A Unity chama essas funções sozinha quando o player encosta ou deixa de encostar em algo.
- Só conta objetos com a tag `Plataforma`. Encostar numa parede sem a tag não recarrega o pulo.
- Ao sair, tira 1, mas nunca deixa ficar negativo.

Resumo: o caminho do script

- **Start**: prepara o Rigidbody, o cursor e o checkpoint. Roda uma vez.
- **Update**: gira a câmera, recarrega os pulos no chão e faz o pulo. Roda a cada frame.
- **FixedUpdate**: lê WASD, decide o estado e muda a velocidade aos poucos. Roda 50 vezes por segundo.
- **Colisões**: contam as plataformas que o player está tocando. A Unity chama sozinha.